

SUPPLEMENTARY PROBLEM: LONGEST INCREASING SUBSEQUENCE

Instructor: Orhan Özgüner

© Özgüner @CWRU, 2025

Problem Statement:

We are given a sequence $A = [a_1, a_2, \dots, a_n]$ of n numbers. We would like to find the longest *not necessarily consecutive* subsequence of A such that the numbers in the subsequence are in strictly increasing order, i.e., we want to find the maximum k such that there exists a subsequence $[a_{i_1}, a_{i_2}, \dots, a_{i_k}]$ of A with $i_1 < i_2 < \dots < i_k$ and $a_{i_1} < a_{i_2} < \dots < a_{i_k}$.

Example:

- Input: $n = 7$, $A = [7, 3, 4, 1, 6, 8, 2]$.
- Output: The longest increasing subsequence (LIS) of A is $[3, 4, 6, 8]$ with length 4.

Develop an efficient solution to solve this problem.

Here are guided steps to help you while thinking about a solution (You do not have to follow these steps):

1. Identify the “last” decision you need to make.
2. Define subproblems, i.e., describe the table you need to compute.
3. Provide a recursive formulation to compute the entries of the table.
4. Explain how you would reconstruct the optimal solution based on the computed table.
5. Run through an example to show how your algorithm works.
6. Prove the correctness of your algorithm.
7. Analyze the runtime of your algorithm assuming bottom-up computation of the table.

Solution

1: The “last” decision

We want to build up the solution so that we can decide which element of A should be the final element in the LIS.

2: Define subproblems

Consider the subarray $A[1, \dots, i]$. We define:

- d_i : Length of the longest increasing subsequence in $A[1, \dots, i]$, for $0 \leq i \leq n$. We will call the last element in that sequence, a_i .
- p_i : The index of the last element before a_i , in the LIS in $A[1, \dots, i]$ for $0 \leq i \leq n$.

3: Develop recursive formulation to solve subproblems

For convenience, let $a_0 = -\infty$.

$$d_i = \begin{cases} 0 & \text{if } i = 0 \\ 1 + \max_{j < i, a_j < a_i} (d_j) & \text{otherwise} \end{cases} \quad (1)$$

$$p_i = \begin{cases} \text{NIL} & \text{if } i = 0 \\ \operatorname{argmax}_{j < i, a_j < a_i} (d_j) & \text{otherwise} \end{cases} \quad (2)$$

The length of the LIS of A is given by:

$$\text{length}(\text{LIS}(A)) = \max_{1 \leq i \leq n} (d_i) \quad (3)$$

We can implement this formulation in a bottom-up manner as follows:

Algorithm 1 Compute Longest Increasing Subsequence

```
1: function COMPUTELIS( $A$ )
2:    $d, p \leftarrow$  Array of length  $n$ 
3:   for  $i \leftarrow 1$  to  $n$  do
4:      $d[i], p[i] \leftarrow \text{RECURRELATION}(A, d, p, i)$  ▷ Compute value using recurrence relation
5:   end for
6:   return RECONSTRUCT( $A, d, p, n$ ) ▷ Reconstruct the optimal solution
7: end function
```

Here, the procedure $\text{RECURRELATION}(A, d, p, i)$ implements Equations (1) and (2).

The procedure $\text{RECONSTRUCT}(A, d, p, n)$ uses the table p to reconstruct the longest increasing subsequence of A .

4: Reconstruct the optimal solution

The index of the last element in the LIS of A is given by:

$$r = \underset{1 \leq i \leq n}{\operatorname{argmax}} (d_i) \quad (4)$$

Thus, to reconstruct the LIS of A , we can initialize our subsequence with r and recursively append $r = p[r]$ to the front of our subsequence until we have $r = \text{NIL}$.

Algorithm 2 Reconstruct the longest increasing subsequence

```

1: function RECONSTRUCT( $A, d, p, n$ )
2:   Initialize empty sequence  $S$ 
3:    $r \leftarrow \underset{1 \leq i \leq n}{\operatorname{argmax}} (d[i])$ 
4:   while  $r \neq \text{NIL}$  do
5:     Append  $A[r]$  to the front of  $S$ 
6:      $r \leftarrow p[r]$ 
7:   end while
8:   return  $S$ 
9: end function

```

5: Show execution on an example

Consider the example given in the problem statement: $A = [7, 3, 4, 1, 6, 8, 2]$.

The following tables should be computed progressively on the board:

i	0	1	2	3	4	5	6	7
$A[i]$	$-\infty$	7	3	4	1	6	8	2
$d[i]$	0	1	1	2	1	3	4	2
$p[i]$	NIL	NIL	NIL	2	NIL	3	5	4

(5)

Thus, the length of the LIS of A is $d[6] = 4$, since it is the largest number in table d .

Since the largest entry of d is the 6th element of d , we know that the last element in the LIS is the 6th element of A . So we can reconstruct the LIS recursively as follows:

- $S = A[6] = 8$
- $p[6] = 5$, thus append $A[5] = 6$ to the front: $S = [6, 8]$
- $p[5] = 3$, thus append $A[3] = 4$ to the front: $S = [4, 6, 8]$
- $p[3] = 2$, thus append $A[2] = 3$ to the front: $S = [3, 4, 6, 8]$
- $p[2] = \text{NIL}$, thus stop and return $S = [3, 4, 6, 8]$ as the LIS.

6: Prove correctness

The algorithm finds the LIS of $A[1, \dots, i]$ for all $1 \leq i \leq n$, and the corresponding last elements a_i . Then, it picks the longest out of these (Equation 4). Since one of the elements in A has to be the last element in the LIS, this guarantees that the algorithm will find the correct LIS, provided that the LIS of each $A[1, \dots, i]$ is computed correctly.

To compute the LIS for $A[1, \dots, i]$ (where a_i is the last element of the LIS), the algorithm considers the LIS of all prefixes of $A[1, \dots, i]$ such that the last element of the prefix is less than a_i . Namely, the set $\{j < i, a_j < a_i\}$ contains all possible prefixes of $A[1, \dots, i]$ whose LIS can be extended with a_i to obtain an LIS for the whole $A[1, \dots, i]$. The algorithm picks the longest out of these and appends a_i to obtain an LIS for $A[1, \dots, i]$ (Equation 1). Consequently, the correct LIS is computed for $A[1, \dots, i]$ provided that the correct LIS is computed for the prefixes of $A[1, \dots, i]$.

Since the base case LIS is trivially correct (the length of the LIS of a zero-length sequence is 0), we conclude by induction that the algorithm correctly computes the LIS of A .

7: Analyze runtime

We can compute the tables d and p for $i = 1$ to n in a bottom-up manner. To compute d_i (and p_i), we perform $\Theta(i)$ work- as the size of the set $\{j < i, a_j < a_i\}$ (the set of the candidates for the last element before a_i), can be as large as $i - 1$. Thus the runtime of the table computation with a bottom-up algorithm is as follows:

$$T(n) = \sum_{i=1}^n \Theta(i) = \Theta(n^2) \tag{6}$$

Since reconstructing the optimal solution makes one pass over the array d and then another pass over the array r , it takes $\Theta(n)$ time.

Thus, the total runtime of the algorithm is $\Theta(n^2) + \Theta(n) = \Theta(n^2)$.